

## Machine Learning Essentials

### Exercise Sheet 09

**Due: 07.07.2025 11:15**

---

On Sheet 8, we discovered that kernelized Bayesian linear regression provides a direct bridge to the **function-space view** of ML. On this sheet, we fully embrace this perspective by exploring **Gaussian Processes (GPs)**. We'll first cover some of the nuts and bolts of applying GPs for regression, focusing on how kernels can be combined and how their hyperparameters can be learned from data. We then move to a powerful application of GPs: **Bayesian Optimization (BO)**, a method for efficiently optimizing expensive black-box functions. You'll implement BO to find optimal hyperparameter settings for a neural network and apply the similar principles of **Bayesian search** to a problem inspired by a real-world salvaging operation.

#### Regulations

Please submit your solutions via Moodle in teams of **3** students. The coding tasks must be completed using the templates `MLE25_sheet09_ex1.3.ipynb`, `MLE25_sheet09_ex2.ipynb` and `MLE25_sheet09_ex3.ipynb`. Each submission must include **exactly** one file:

- Upload a `.pdf` file containing both your Jupyter notebook and solutions to analytical exercises.

The Jupyter notebook can be exported to pdf by selecting **File** → **Download as** → **pdf** in JupyterLab. If this method does not work, you may print the notebook as a pdf instead. Your analytical solutions can be either scanned handwritten solutions or created using  $\text{\LaTeX}$ .

### Exercise 1: Nuts and Bolts of Gaussian Process Regression

As introduced in the lecture, a GP is a collection of random variables, any finite number of which have a joint Gaussian distribution. It is fully specified by a **mean function**  $m(\mathbf{x})$  and a **covariance function**  $k(\mathbf{x}, \mathbf{x}')$ . We saw on Sheet 8 that the kernel (here: covariance function) contains our prior beliefs about the properties (e.g. smoothness) of the function we wish to model. We'll now discuss some practical aspects of **how to “engineer” these kernels and choose proper hyperparameters** for them.

## Tasks

1. In the lecture it was discussed that a function is a **valid kernel** iff its corresponding **Gram/Kernel matrix**  $K_{ij} = k(\mathbf{x}_i, \mathbf{x}_j)$  is **positive semi-definite (psd)** for all choices of input points. From this, a powerful feature of GPs arises: The ability to create complex kernels by combining simpler ones. The corresponding **Kernel algebra** contains all operations that preserve the psd-ness of the original kernel matrix. Examples include: Taking the product of two kernels, scaling a kernel by a positive constant, or exponentiating a kernel.

Given two valid kernel functions,  $k_1(\mathbf{x}, \mathbf{x}')$  and  $k_2(\mathbf{x}, \mathbf{x}')$ , show that their sum

$$k_{\text{sum}}(\mathbf{x}, \mathbf{x}') = k_1(\mathbf{x}, \mathbf{x}') + k_2(\mathbf{x}, \mathbf{x}')$$

is also a valid kernel.

(1 pts.)

2. As kernels come with their own hyperparameters (e.g. lengthscale), the question arises of **how to choose these hyperparameters** in a principled way. Intuitively, a good model should be one that balances fitting the data with not being overly complex. In a Bayesian framework, **model selection** incorporates this notion in terms of the **evidence** or **marginal likelihood**, which essentially quantifies how good a model  $\mathcal{M}$  fits the data  $\mathcal{D}$ , averaged over all possible parameter settings, weighted by the prior probability of those parameters:

$$p(\mathcal{D}|\mathcal{M}) = \int p(\mathcal{D}|\mathbf{w}, \mathcal{M})p(\mathbf{w}|\mathcal{M})d\mathbf{w}.$$

In contrast to MLE/MAP, averaging over the prior  $p(\mathbf{w}|\mathcal{M})$  naturally penalizes overly complex models that might fit the data well for specific parameter settings but poorly for most. This is often called the **“automatic Occam’s Razor”** of Bayesian model selection. Let’s now say we want to learn the hyperparameters  $\boldsymbol{\theta}$  of a GP’s covariance function from data. We can do so by **maximizing the log marginal likelihood**. This is often called **Type-II maximum likelihood** (to distinguish it from standard MLE).

- (a) For observations with i.i.d Gaussian noise,  $\epsilon_i \sim \mathcal{N}(0, \sigma^2)$ , show that for a GP with  $m(\mathbf{x}) = 0$ , the log marginal likelihood is given by

$$\log p(\mathbf{y}|\mathbf{X}, \boldsymbol{\theta}) = \underbrace{-\frac{1}{2}\mathbf{y}^\top (\mathbf{K}_{\boldsymbol{\theta}} + \sigma^2 \mathbf{I}_n)^{-1} \mathbf{y}}_{\text{Data Fit}} \underbrace{-\frac{1}{2} \log |\mathbf{K}_{\boldsymbol{\theta}} + \sigma^2 \mathbf{I}_n|}_{\text{Complexity Penalty}} \underbrace{-\frac{n}{2} \log(2\pi)}_{\text{Const.}}$$

**Hint:** Start with the model  $\mathbf{y} = \mathbf{f} + \boldsymbol{\epsilon}$ , where  $\mathbf{f}$  are the true (unobserved)

function values, modeled by the GP. Marginalize the likelihood  $p(\mathbf{y}|\mathbf{X})$  over all possible  $\mathbf{f}$  - this yields a Gaussian (recall the last Sheet), whose mean and covariance you can easily compute. From this you can formulate the corresponding Gaussian pdf and get the log marginal likelihood.

(2 pts.)

- (b) Explain the role of the “Data Fit” and “Complexity Penalty” terms. Why is the second term an “Occam’s razor”?

(1 pts.)

3. Now let’s see some “kernel engineering” in practice. The **Mauna Loa CO2 dataset** is a famous **time series** benchmark dataset. It contains the longest continuous record of atmospheric carbon dioxide measurements, primarily collected at the Mauna Loa Observatory in Hawaii. It’s also known as the **Keeling curve**, after the scientist who started collecting the data<sup>1</sup>. The time series exhibits several distinct patterns: a long-term increasing trend, a yearly periodic cycle, and smaller, medium to short-term irregularities. Your goal is to **perform GP regression on this data** by constructing a composite kernel tailored to these patterns. You are not expected to get a “perfect” model here, but justify your choices.

- (a) Load the CO2 dataset using the provided code.
- (b) Using the modern GP library **GPYtorch**, design a kernel that captures the different patterns of the time series. A popular “guide” to combining and interpreting kernels is the “Kernel Cookbook”. Some hints can also be found in the notebook. To see whether your kernel is reasonable, you can draw a few samples from the GP prior and compare these samples against the data.

(4 pts.)

- (c) Use the Adam optimizer and stochastic gradient descent to find the kernel hyperparameters by minimizing the negative log marginal likelihood.

(1 pts.)

- (d) Use the provided code to produce a plot showing (i) the training data, (ii) the posterior mean of your fitted GP, (iii) the 95% confidence interval, and (iv) an extrapolation of a few years into the past/future. Assess your results. Does your GP extrapolate well? Why/why not? Print the learned hyperparameters of your GP and discuss whether they correspond to the physical characteristics you would expect of the Keeling curve (e.g. a yearly cycle due to seasonal growth and decay of vegetation).

(2 pts.)

---

<sup>1</sup>Charles Keeling did maintain this dataset starting from 1958 until his death in 2005. Afterwards, his son Ralph Keeling continued leading the measurement program. Coincidentally, he’s giving a talk about the measurement program on 18th July at the “Physikalisches Kolloquium” in Heidelberg, if you’re interested.

## Exercise 2: Bayesian Optimization for Hyperparameter Tuning

Training a neural network can be computationally expensive. Finding the best hyperparameters (e.g. learning rate, regularization strength) via a brute-force grid search is often inefficient or infeasible. **Bayesian Optimization (BO)** is a sample-efficient technique that can be applied to this task. In general, BO can be used for **maximizing expensive-to-evaluate black-box functions** (here: the validation accuracy) using a probabilistic surrogate model (most often a GP). BO relies on defining an **acquisition function** that makes use of the model’s predictions and its uncertainty to intelligently decide which input (here: hyperparameter setting) to evaluate next. This is done by navigating the **exploration-exploitation** tradeoff between exploring new, unseen regions of the black-box function and exploiting regions of known large values.

### Tasks

For this exercise, we’ll use the **HTRU2 dataset** to search for pulsar stars. Pulsars are a rare type of rotating neutron star that are of great scientific interest to astrophysicists because they are natural laboratories for extreme physics<sup>2</sup>.

Your task is to perform hyperparameter tuning for a classifier that can distinguish real pulsar signals from noise, based on 8 statistical features derived from radio telescope data. You will do so by implementing a BO model to find the best **learning rate** and **batch size** for a simple **MLP** trained on this dataset.

1. Given the defined model and training loop on the notebook, define your black-box objective function. This function should take a 2D hyperparameter vector (**learning\_rate**, **batch\_size**) as input, configure and train a simple MLP for a few epochs, and return the final validation accuracy.

(2 pts.)

2. First, perform a brute-force grid search over a reasonable range of the two hyperparameters (e.g. using the pre-defined  $10 \times 10$  grid, or a smaller one if it takes too long). Store the results and visualize the performance landscape using a heatmap. This will serve as our “ground truth”.

(1 pts.)

3. Now, we perform Bayesian Optimization:

- (a) Start by evaluating 5 randomly chosen hyperparameter configurations. These will serve as the training data for the GP (it’s “starting point”).

---

<sup>2</sup>Because their rotational periods are incredibly stable, pulsars act as ultra-precise cosmic clocks. By timing these “clocks”, astrophysicists can e.g. test the predictions of general relativity in extreme conditions.

(b) Implement the BO loop. **In each iteration:**

- Fit a GP surrogate model to the hyperparameter-accuracy pairs observed so far. Use a `gpytorch.kernels.MaternKernel` with  $\nu = 2.5$ .
- Maximize an acquisition function over the search space to select the next point to evaluate. A commonly used acquisition function is the **Upper Confidence Bound (UCB)**:

$$\alpha_{\text{UCB}}(\mathbf{x}) = \mu_{\mathcal{GP}}(\mathbf{x}) + \kappa \sigma_{\mathcal{GP}}(\mathbf{x}),$$

where  $\kappa$  is a tunable parameter. UCB works by selecting the point with the most “optimistic” potential. It balances two competing goals:

- **Exploitation:** Searching in areas where the model predicts a high mean performance ( $\mu_{\mathcal{GP}}$ ). This is where we’d expect to find good solutions.
- **Exploration:** Searching in areas where the model is most uncertain ( $\sigma_{\mathcal{GP}}$ ). This is where we might discover unexpectedly good solutions.

The parameter  $\kappa$  thus controls how much we favor exploration over exploitation.

- Call your expensive objective function at the selected point and add the new data to your observed set.

(c) Run the BO loop for a small budget (e.g. 20 evaluations).

(1 + 3 + 1 pts.)

4. The provided code produces a short video that shows the path of the BO process on the “accuracy surface” provided by the evaluated grid, as well as a comparison between the final results of grid search vs. BO. Explain the visible behavior of the BO process and comment on the results.

(3 pts.)

## Exercise 3: Searching Flight 447

**Note:** This exercise builds a bit on the other ones, so it is recommended you do Exercise 1 and 2 first.

In 2009, Air France Flight 447 tragically crashed into the Atlantic Ocean. After nearly two years of difficult and often fruitless searching, the black boxes were finally located. This success was not just luck, but the result of applying a statistical approach known as **Bayesian Search Theory**<sup>3</sup>. This approach uses prior information (e.g. from flight path simulations and previous search attempts) to continually update a probability map of the

---

<sup>3</sup>For a more precise account of the search for AF447, see for example the paper “Search for the Wreckage of Air France Flight AF 447”.

wreckage's location. These techniques were famously first developed to find a lost hydrogen bomb after a B-52 crash off the coast of Palomares, Spain, in 1966.

In this exercise, you'll implement a simplified 2D version for searching Flight 447. The goal therefore is to find a hidden target on a grid by sequentially choosing where to "search" (much like in BO - you can interpret BO as a specific type of Bayesian search, with a different goal). You'll make use of a GP to model your belief about the target's location.

## Tasks

In this scenario, you're given an "ocean" in the form of a 2D grid. A "wreckage" is hidden at an unknown location. You're also given a "prior belief map" (a 2D Gaussian blob) representing the initial, uncertain probability of the wreckage location. You have access to a `search(x, y)` function that simulates a noisy sonar reading at a given coordinate. A more distant position returns lower values, while searches near the wreckage return a higher signal amplitude. **In BO terms:** Our black-box function  $f(\mathbf{x})$  is the true probability of finding the wreckage at some 2D coordinates  $\mathbf{x}$ . A "search" at some location  $\mathbf{x}_i$  gives us a noisy measurement  $y_i = f(\mathbf{x}_i) + \epsilon_i$  of this probability. Our goal is to find  $\arg \max_{\mathbf{x}} f(\mathbf{x})$ .

1. **Design a GP surrogate model over the 2D search space.** Implement a **mean function** for the GP prior that uses the provided **prior belief map**.

(2 pts.)

2. In a loop, for a fixed number of 30 search attempts:

- (a) Use your GP's posterior to decide the best location to search next. This requires **selecting and implementing an acquisition function**. UCB is a strong candidate, but you could also consider others. Justify your choice.
- (b) **Perform the "search"** by calling the provided `search(x, y)` function with your chosen coordinates.
- (c) Update your GP with the new data point. The code keeps track of the GP's posterior mean (your updated belief map) at each step.

(1+1+1 pts.)

3. While the code provides an animation that shows your Bayesian search agent and your changing belief map "in action", also **provide a static visualization of the search process**. Do this in form of a plot that shows your final belief map alongside the locations you searched and the location of the wreckage location.

(2 pts.)

4. Briefly assess your results.

(1 pts.)